

## **PROGRAMMING WITH C++**

### **UNIT 5**

### **POLYMORPHISM, OVERLOADING, AND SMART POINTERS**

# Outlines

- Introduction to Polymorphism
- Function overloading
- overloading unary and binary operators and type conversion
- Relationships among Objects in an Inheritance Hierarchy
- Virtual Functions and Virtual Destructors
- Abstract Classes and Pure virtual Functions
- Runtime Polymorphism and Compile-Time Polymorphism
- Operator Overloading Fundamentals
- Dynamic Memory Management with new and delete
- Resource acquisition is initialization (RAII) and Smart Pointers
- Three-Way Comparison Operator(< = >)
- Converting Between Types
- Explicit Constructors and Conversion Operators
- Overloading the Function Call Operator ()

# Introduction to Polymorphism in C++

- Polymorphism is one of the four key principles of Object-Oriented Programming (OOP) in C++.
- It allows objects of different classes to be treated as objects of a common base class.
- Polymorphism enables flexibility and scalability in code by allowing a single interface to control multiple types of behavior.
- **Types of Polymorphism in C++**  
Polymorphism in C++ is categorized into two main types:
  - **Compile-time Polymorphism (Static Binding)**
    - Achieved using **function overloading** and **operator overloading**.
    - Resolved at compile time.
  - **Run-time Polymorphism (Dynamic Binding)**
    - Achieved using **virtual functions** and **base class pointers/references**.
    - Resolved at runtime.

# Key Advantages of Polymorphism

- **Code reusability:** A single interface can be used for multiple implementations.
- **Extensibility:** Adding new functionality is easier without modifying existing code.
- **Maintainability:** Reduces code complexity by promoting modular design.

# Compile-time Polymorphism

- Compile-time polymorphism, also known as **static polymorphism**, occurs when the function call is resolved at **compile time** rather than at runtime. This is achieved using:
  - **Function Overloading**
  - **Operator Overloading**

# Function Overloading

- Function overloading allows multiple functions to have the **same name** but different **parameters** (number, type, or both).
- The compiler determines which function to call based on the arguments passed.

# Example of Function Overloading

```
#include <iostream>
using namespace std;

class Math {
public:
    // Overloaded function: same name, different parameters
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
};
```

```
int main() {
    Math obj;
    cout << obj.add(5, 3) << endl;    // Calls int version
    cout << obj.add(2.5, 3.7) << endl; // Calls double version
    cout << obj.add(1, 2, 3) << endl; // Calls three-parameter version
    return 0;
}
```

**Output:**

```
8
6.2
6
```

# Operator Overloading

- Operator overloading allows customizing the behavior of operators for user-defined types (like classes). This helps in making objects interact naturally using operators like +, -, \*, etc., similar to primitive types.
- Types of Operators:
  - Unary Operators (+, -, ++, --)
  - Binary Operators (+, -, \*, /, %, ==, !=)
  - Special Operators ([], (), ->, =)

## Syntax:

```
return_type operator<symbol>(arguments)
{
    // Custom implementation
}
```



# Overloading Binary Operators

```
#include <iostream>
using namespace std;

class Complex {
private:
    int real, imag;
public:
    Complex(int r = 0, int i = 0)
    {
        real=r;
        imag=i;
    }

    // Overloading the + operator
    Complex operator+(Complex obj) {
        return Complex(real + obj.real, imag + obj.imag);
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3, 2), c2(1, 7);
    Complex c3 = c1 + c2; // Calls the overloaded operator+
    c3.display();
    return 0;
}
```

**Output:**

4 + 9i

# Overloading Unary Operators

```
#include <iostream>
using namespace std;

class Count {
private:
    int value;
public:
    Count(int v = 0) : value(v) {}

    // Overloading prefix ++
    void operator++() {
        ++value;
    }
    // Overloading postfix ++
    void operator++(int) {
        value++;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Count c(5);
    ++c; // Calls prefix ++
    c.display();

    c++; // Calls postfix ++
    c.display();
    return 0;
}
```

**Output:**

```
Value: 6
Value: 7
```

# Run-time Polymorphism

- Run-time polymorphism is achieved using **function overriding** and **virtual functions** in C++.
- It allows a **derived class** to provide a specific implementation of a function that is already defined in its **base class**.
- The function call is resolved **at runtime** using **dynamic binding** (late binding).

# Function Overriding (Virtual Functions)

- Function overriding allows a derived class to provide a specific implementation of a function that is already defined in its base class.
- The base class function is marked as virtual to enable dynamic.
- Virtual functions enable dynamic dispatch, meaning the function that gets executed is determined at runtime.

# Function Overriding Example

```
// C++ program to demonstrate compile time  
function overriding  
  
#include <iostream>  
using namespace std;  
  
class Parent {  
public:  
    void GeeksforGeeks_Print()  
    {  
        cout << "Base Function" << endl;  
    }  
};  
  
class Child : public Parent {  
public:  
    void GeeksforGeeks_Print()  
    {  
        cout << "Derived Function" << endl;  
    }  
};  
  
int main()  
{  
    Child Child_Derived;  
    Child_Derived.GeeksforGeeks_Print();  
    return 0;  
}
```

**Output:**

Derived Function

# Function Overriding (Virtual Functions) Example

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() { // Virtual function
        cout << "Animal makes a sound." << endl;
    }
};

class Dog : public Animal {
public:
    void makeSound() override { // Overriding function
        cout << "Dog barks." << endl;
    }
};

class Cat : public Animal {
public:
    void makeSound() override { // Overriding function
        cout << "Cat meows." << endl;
    }
};
```

```
int main() {
    Animal* a;    // Pointer to base class
    Dog d;
    Cat c;

    a = &d;
    a->makeSound(); // Calls Dog's makeSound() due to dynamic binding

    a = &c;
    a->makeSound(); // Calls Cat's makeSound() due to dynamic binding

    return 0;
}
```

**Output:**

```
Dog barks.
Cat meows.
```

# Pure Virtual Functions & Abstract Classes

- A **pure virtual function** is a function in a **base class** that **must be overridden** by **derived classes**.
- It makes the base class an **abstract class**, meaning it **cannot be instantiated**.

## Syntax:

```
class Base {  
public:  
    virtual void functionName() = 0; // Pure virtual function  
};
```

# Pure Virtual Functions & Abstract Classes Example

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0; // Pure virtual function (abstract class)
};

class Circle : public Shape {
public:
    void draw() override { // Overriding pure virtual function
        cout << "Drawing a Circle." << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() override { // Overriding pure virtual function
        cout << "Drawing a Rectangle." << endl;
    }
};

int main() {
    Shape* s; // Pointer to base class

    Circle c;
    Rectangle r;

    s = &c;
    s->draw(); // Calls Circle's draw()

    s = &r;
    s->draw(); // Calls Rectangle's draw()

    return 0;
}
```

**Output:**

```
Drawing a Circle.
Drawing a Rectangle.
```



# C++ Type Conversion

- Type conversion in C++ refers to converting one data type into another. It can be categorized into **implicit** and **explicit** type conversion.

| Type Conversion                               | Description                          | Example                                                           |
|-----------------------------------------------|--------------------------------------|-------------------------------------------------------------------|
| Implicit Conversion                           | Automatic type conversion            | <code>int x = 10; double y = x;</code>                            |
| C-Style Cast<br><code>(type)variable</code>   | Simple but unsafe                    | <code>int a = (int)3.14;</code>                                   |
| <code>static_cast&lt;T&gt;(value)</code>      | Safe and preferred                   | <code>double d = 9.9; int i = static_cast&lt;int&gt;(d);</code>   |
| <code>reinterpret_cast&lt;T&gt;(value)</code> | Converts pointer types               | <code>char* ptr = reinterpret_cast&lt;char*&gt;(&amp;num);</code> |
| <code>const_cast&lt;T&gt;(value)</code>       | Removes <code>const</code> qualifier | <code>const int* p; int* q = const_cast&lt;int*&gt;(p);</code>    |
| <code>dynamic_cast&lt;T&gt;(value)</code>     | Used for safe downcasting in OOP     | <code>Derived* d = dynamic_cast&lt;Derived*&gt;(b);</code>        |

# Implicit Type Conversion (Type Promotion)

- Also known as **automatic type conversion**, this happens when the compiler automatically converts one data type into another without explicit intervention by the programmer.

## Rules for Implicit Conversion

- Conversion happens when assigning a value of a smaller data type to a larger data type.
- There is no loss of data in integer promotions or widening conversions.
- Data loss may occur in narrowing conversions.

# Example

```
#include <iostream>
using namespace std;

int main() {
    int num = 10;
    double dbl = num; // Implicit conversion from int to double

    cout << "num (int) = " << num << endl;
    cout << "dbl (double) = " << dbl << endl; // Output: 10.0

    return 0;
}
```

**Output:**

```
num (int) = 10
dbl (double) = 10
```

# Explicit Type Conversion (Type Casting)

- Explicit conversion requires the programmer to specify the type conversion manually using **type casting**. There are three ways to perform explicit type conversion in C++:
  - **C-Style Cast**
  - **C++ Static Cast (`static_cast`)**
  - **Other C++ Casts**
    - C++ provides other type casting methods for specific use cases:
      - `dynamic_cast` (for polymorphic class conversions)
      - `const_cast` (to add/remove `const` from variables)
      - `reinterpret_cast` (for low-level pointer manipulation)

# C-Style Cast ((type)variable)

```
#include <iostream>
using namespace std;

int main() {
    double num = 9.99;
    int x = (int)num; // Explicit conversion using C-style cast

    cout << "Original value (double): " << num << endl;
    cout << "Converted value (int): " << x << endl; // Output: 9

    return 0;
}
```

## Output:

```
Original value (double): 9.99
Converted value (int): 9
```

# static\_cast<> (Recommended in C++)

```
#include <iostream>
using namespace std;

int main() {
    double num = 9.99;
    int x = static_cast<int>(num); // Explicit conversion using static_cast

    cout << "Original value (double): " << num << endl;
    cout << "Converted value (int): " << x << endl; // Output: 9

    return 0;
}
```

**Output:**

```
Original value (double): 9.99
Converted value (int): 9
```

# reinterpret\_cast<> (For Pointer Conversion)

```
#include <iostream>
using namespace std;

int main() {
    int num = 65;
    char* ptr = reinterpret_cast<char*>(&num); // Reinterpreting int memory as char*

    cout << "Reinterpreted character: " << *ptr << endl; // Output: 'A' (ASCII 65)

    return 0;
}
```

**Output:**

```
Reinterpreted character: A
```

# const\_cast<> (Removing const Qualifier)

```
#include <iostream>
using namespace std;

void modify(int* ptr) {
    *ptr = 42;
}

int main() {
    const int num = 10;
    int* ptr = const_cast<int*>(&num); // Removing const
    modify(ptr);

    cout << "Modified value: " << *ptr << endl; // Output: 42 (Undefined behavior warning!)

    return 0;
}
```

**Output:**

Modified value: 42



# dynamic\_cast<> (For Polymorphic Classes)

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {} // Virtual function makes Base
    polymorphic
};

class Derived : public Base {};

int main() {
    Base* basePtr = new Derived();
    Derived* derivedPtr = dynamic_cast<Derived*>(basePtr);
    // Safe downcasting

    if (derivedPtr) {
        cout << "Successfully casted to Derived!" << endl;
    } else {
        cout << "Failed to cast!" << endl;
    }

    delete basePtr;
    return 0;
}
```

**Output:**

Successfully casted to Derived!

# User-Defined Type Conversion

- C++ allows defining custom type conversions in **classes** by using:
  - **Conversion constructors**
  - **Conversion operators**

# Example: Conversion Constructor

```
#include<iostream>
using namespace std;
class MyClass {
public:
    int value;
    MyClass(int v) { value = v; } // Conversion constructor
};

int main() {
    MyClass obj = 100; // Implicit conversion from int to MyClass
    cout << obj.value; // Output: 100
}
```

**Output:**

A terminal window with a dark background showing the output '100' in a light blue, monospaced font.

# Example: Conversion Operator

```
#include<iostream>
using namespace std;
class MyClass {
public:
    int value;
    MyClass(int v) : value(v) {}

    operator int() { // Conversion operator
        return value;
    }
};

int main() {
    MyClass obj(200);
    int x = obj; // Implicit conversion from MyClass to int
    cout << x; // Output: 200
}
```

**Output:**

200

# Relationships among Objects in an Inheritance Hierarchy

- Inheritance in C++ establishes relationships among objects based on a parent-child hierarchy. It allows a class (derived class) to inherit properties and behaviors from another class (base class). This promotes code reuse, abstraction, and polymorphism.

| Relationship Type        | Description                                                       | Example                                  |
|--------------------------|-------------------------------------------------------------------|------------------------------------------|
| IS-A                     | A derived class is a more specialized form of a base class.       | Car is a Vehicle                         |
| HAS-A                    | A class contains another class as a member.                       | Car has an Engine                        |
| POLYMORPHISM             | A derived class can override base class behavior dynamically.     | Animal class → Dog overrides makeSound() |
| MULTIPLE INHERITANCE     | A class inherits from multiple base classes.                      | Car inherits from Engine and Wheels      |
| HIERARCHICAL INHERITANCE | Multiple derived classes inherit from a common base class.        | Dog & Cat inherit from Animal            |
| HYBRID INHERITANCE       | Combination of multiple inheritance and hierarchical inheritance. | FlyingCar inherits from Car & Airplane   |

# IS-A Relationship (Generalization & Specialization)

- Represents "is a type of" relationship.
- A derived class is a specialized form of its base class.

## **Example:**

- A Car is a Vehicle.
- A Dog is an Animal.

# Example

```
#include <iostream>
using namespace std;

class Vehicle { // Base class
public:
    void show() { cout << "I am a Vehicle" << endl; }
};

class Car : public Vehicle { // Derived class (Car is a Vehicle)
public:
    void display() { cout << "I am a Car" << endl; }
};

int main() {
    Car myCar;
    myCar.show(); // Inherited method
    myCar.display(); // Car-specific method
    return 0;
}
```

**Output:**

```
I am a Vehicle
I am a Car
```

# HAS-A Relationship (Composition/Aggregation)

- Represents "has a" relationship.
- One class contains an instance (or pointer) of another class.

## **Example:**

- A Car has a Engine.
- A Company has a Employee.



# Example

```
#include <iostream>
using namespace std;

class Engine {
public:
    void start() { cout << "Engine Started" << endl; }
};

class Car {
private:
    Engine engine; // Car has an Engine
public:
    void startCar() {
        engine.start(); // Using Engine's functionality
        cout << "Car is Moving" << endl;
    }
};

int main() {
    Car myCar;
    myCar.startCar();
    return 0;
}
```

**Output:**

```
Engine Started
Car is Moving
```

# Other Relationships

- **POLYMORPHISM (Dynamic Behavior)**
  - A derived class can override the behavior of a base class.
  - Achieved via **function overriding and virtual functions**.
- **MULTIPLE INHERITANCE (Multiple Parent Classes)**
  - A class can inherit from **more than one base class**.
  - Helps in combining functionalities from multiple sources.
- **HIERARCHICAL INHERITANCE (Common Base Class)**
  - Multiple derived classes inherit from a single base class.
- **HYBRID INHERITANCE (Combination of Types)**
  - Mixes multiple inheritance with hierarchical inheritance.

# Virtual Functions

- A virtual function is a member function of a base class that can be overridden in a derived class and supports dynamic binding (runtime polymorphism).
- When a function is marked as virtual, C++ determines which function to call at runtime instead of compile-time.

## **Key Features of Virtual Functions**

- Declared in the base class using the virtual keyword.
- Allows function overriding in the derived class.
- Supports dynamic binding via base class pointers or references.
- Enables polymorphism (calling derived class methods through base class pointers).

# Example

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() { // Virtual function
        cout << "Base class show()" << endl;
    }
};

class Derived : public Base {
public:
    void show() override { // Override the virtual function
        cout << "Derived class show()" << endl;
    }
};
```

```
int main() {
    Base* basePtr; // Base class pointer
    Derived obj;
    basePtr = &obj;

    basePtr->show(); // Calls Derived class show() due to
dynamic binding

    return 0;
}
```

**Output:** Derived class show()

# Virtual Destructors

- A **virtual destructor** ensures that the destructor of a **derived class** is **called properly** when deleting an object through a **base class pointer**.
- **Why Do We Need a Virtual Destructor?**
  - Without a virtual destructor, **only the base class destructor** runs, leading to **memory leaks** when a derived class dynamically allocates memory.

| Feature        | Virtual Function                          | Virtual Destructor                                              |
|----------------|-------------------------------------------|-----------------------------------------------------------------|
| Purpose        | Enables runtime polymorphism              | Ensures proper cleanup of derived objects                       |
| Declared Using | <code>virtual</code> before function name | <code>virtual</code> before destructor ( <code>~Base()</code> ) |
| Behavior       | Overridden in derived class               | Ensures derived class destructor runs first                     |
| When Used?     | Function overriding in inheritance        | Deleting objects through base class pointers                    |

# Example

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual ~Base() { cout << "Base Destructor" << endl; }
};

class Derived : public Base {
public:
    ~Derived() { cout << "Derived Destructor" << endl; }
};

int main() {
    Base* ptr = new Derived();
    delete ptr; // Calls both Base and Derived destructors

    return 0;
}
```

**Output:**

```
Derived Destructor
Base Destructor
```

# Abstract Classes and Pure Virtual Functions

- In C++, **abstract classes** and **pure virtual functions** play a crucial role in defining **interfaces** and enabling **polymorphism**.
- They allow us to create **base classes** that define a common structure for derived classes while enforcing the implementation of specific functions.

# Pure Virtual Functions

- A pure virtual function is a function that must be overridden in a derived class. It is declared in a base class with = 0.

**Syntax:**

```
virtual ReturnType FunctionName() = 0;
```

| Feature                      | Abstract Class                                           | Pure Virtual Function                                     |
|------------------------------|----------------------------------------------------------|-----------------------------------------------------------|
| Definition                   | A class that contains at least one pure virtual function | A function declared with <code>= 0</code> in a base class |
| Can be instantiated?         | ✗ No                                                     | N/A                                                       |
| Purpose                      | Acts as a base class for derived classes                 | Enforces function overriding in derived classes           |
| Contains concrete functions? | ✓ Yes, it can                                            | ✗ No, it must be overridden in derived classes            |
| Used for                     | Defining a common interface                              | Enforcing behavior in derived classes                     |



# Example

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() = 0; // Pure virtual function
};

class Dog : public Animal {
public:
    void makeSound() override {
        cout << "Woof! Woof!" << endl;
    }
};
```

```
int main() {
    // Animal a; // ✗ ERROR: Cannot instantiate an abstract
class
    Dog d;
    d.makeSound(); // Output: Woof! Woof!
    return 0;
}
```

**Output:**

**Woof! Woof!**

# Abstract Classes

- An **abstract class** is a class that **cannot be instantiated** on its own and is meant to be a **base class** for other classes.
- It contains at least one **pure virtual function**.
- **Key Features of Abstract Classes**
  - **Cannot** create objects of an abstract class.
  - Used to define a **common interface** for derived classes.
  - **Must** have at least one **pure virtual function**.
  - Derived classes must **implement** all pure virtual functions.

# Example

```
#include <iostream>
using namespace std;

class Shape { // Abstract class
public:
    virtual void draw() = 0; // Pure virtual function
};

class Circle : public Shape {
public:
    void draw() override {
        // Implementing pure virtual function
        cout << "Drawing a Circle" << endl;
    }
};
```

```
int main() {
    // Shape s; // ✗ ERROR: Cannot instantiate an abstract
    class
    Circle c;
    c.draw(); // Output: Drawing a Circle

    return 0;
}
```

**Output:** Drawing a Circle

# Example Abstract Class with Concrete Functions

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void makeSound() = 0; // Pure virtual function
    void breathe() { // Concrete function
        cout << "Breathing..." << endl;
    }
};

class Cat : public Animal {
public:
    void makeSound() override {
        cout << "Meow!" << endl;
    }
};

int main() {
    Cat c;
    c.breathe(); // Output: Breathing... (from base class)
    c.makeSound(); // Output: Meow! (overridden)
    return 0;
}
```

**Output:**

```
Breathing...
Meow!
```

# Dynamic Memory Management with new and delete

- In C++, dynamic memory management allows you to allocate and deallocate memory at runtime using new and delete.
- This is useful when the size of an object is unknown at compile-time or needs to be flexible.

## Why Use Dynamic Memory?

- Memory is allocated **at runtime** (heap memory).
- Enables **dynamic arrays** and **flexible object creation**.
- Prevents **stack overflow** when dealing with large objects.
- Necessary when working with **pointers and polymorphism**.

# Allocating Memory with new

- The new operator allocates memory on the heap and returns a pointer to it.

## Syntax:

```
DataType* pointer = new DataType; // Allocates memory for a single variable  
DataType* arrayPointer = new DataType[size]; // Allocates memory for an array
```

# Deallocating Memory with delete

- The delete operator frees dynamically allocated memory to prevent memory leaks.

## **Syntax:**

```
delete pointer; // Frees memory for a single variable  
delete[] pointer; // Frees memory for an array
```

# Example

```
#include <iostream>
using namespace std;

int main() {
    int* ptr = new int; // Allocates memory for an integer
    *ptr = 42; // Assign value to allocated memory

    cout << "Value: " << *ptr << endl; // Output: 42

    delete ptr; // Free allocated memory
    return 0;
}
```

**Output:**

Value: 42



# Example of Allocating a Single Object

```
#include <iostream>
using namespace std;

class Car {
public:
    void start() {
        cout << "Car is starting..." << endl;
    }
};

int main() {
    Car* myCar = new Car; // Allocates memory for a Car object
    myCar->start(); // Call member function

    delete myCar; // Free allocated memory
    return 0;
}
```

**Output:**

```
Car is starting...
```

# Example of Allocating Arrays of Objects

```
#include <iostream>
using namespace std;

class Car {
public:
    Car() { cout << "Car Created" << endl; }
    ~Car() { cout << "Car Destroyed" << endl; }
};

int main() {
    Car* cars = new Car[3]; // Allocates an array of 3 Car objects

    delete[] cars; // Properly delete the array of objects
    return 0;
}
```

**Output:**

```
Car Created
Car Created
Car Created
Car Destroyed
Car Destroyed
Car Destroyed
```

# Resource Acquisition Is Initialization (RAII) and Smart Pointers

- **Resource Acquisition Is Initialization (RAII)** is a fundamental **memory management** and **resource management** technique in C++.
- It ensures that resources such as **memory, file handles, sockets, and locks** are properly released when an object goes out of scope.
- **Smart pointers** in C++ automate resource management by wrapping raw pointers and ensuring proper cleanup.

# What is RAII?

- **RAII (Resource Acquisition Is Initialization)** is a design pattern where:
  - A **resource** (e.g., heap memory, file, lock) is **acquired in the constructor**.
  - The **resource is released in the destructor** automatically when the object goes out of scope.

# Example

```
#include <iostream>
using namespace std;

class RAII {
private:
    int* data;
    // Pointer to dynamically allocated memory
public:
    RAII(int value) {
        data = new int(value);
        // Resource acquisition (allocating memory)
        cout << "Resource allocated: " << *data << endl;
    }

    ~RAII() {
        delete data; // Resource release (freeing memory)
        cout << "Resource deallocated" << endl;
    }
};
```

```
int main() {
    { // Scope block
        RAII obj(42);
        // Memory is allocated inside the RAII object
    } // `obj` goes out of scope → Destructor is called → Memory is freed

    return 0;
}
```

**Output:**

```
Resource allocated: 42
Resource deallocated
```

# How RAII Works in previous Example

- Constructor allocates memory (`new int(value)`).
- Destructor releases memory (`delete data`).
- When obj goes out of scope, the destructor is called automatically.
- No manual delete is needed, preventing memory leaks.

# Smart Pointers

- C++ smart pointers (`unique_ptr`, `shared_ptr`, `weak_ptr`) in `<memory>` automate RAI for dynamically allocated objects.

| Smart Pointer           | Description                                              | When to Use                                           |
|-------------------------|----------------------------------------------------------|-------------------------------------------------------|
| <code>unique_ptr</code> | Sole ownership of a resource                             | When only <b>one</b> pointer should manage a resource |
| <code>shared_ptr</code> | Shared ownership using reference counting                | When <b>multiple</b> pointers share the resource      |
| <code>weak_ptr</code>   | Non-owning reference to a <code>shared_ptr</code> object | To avoid circular references                          |

# Smart Pointers

- **unique\_ptr: Exclusive Ownership**

- A unique\_ptr ensures **only one** pointer manages an object. When it goes out of scope, it **automatically deletes** the resource.

- **shared\_ptr: Shared Ownership**

- A shared\_ptr **allows multiple pointers** to share ownership of a resource. It uses **reference counting** to track ownership.

- **weak\_ptr: Non-Owning Reference**

- A weak\_ptr is used to avoid **circular references** in shared\_ptr.



# Example of unique\_ptr

```
#include <iostream>
#include <memory> // Required for smart pointers
using namespace std;
```

```
class Car {
public:
    Car() { cout << "Car Created\n"; }
    ~Car() { cout << "Car Destroyed\n"; }
    void drive() { cout << "Driving...\n"; }
};
```

```
int main() {
    unique_ptr<Car> car1 = make_unique<Car>(); // Creates Car object
    car1->drive();
```

```
// unique_ptr does not allow copy
// unique_ptr<Car> car2 = car1; ✗ ERROR
```

```
    unique_ptr<Car> car2 = move(car1);
    // ✓ Transfer ownership using move()
    if (!car1) {
        cout << "car1 is now empty\n";
    }

    return 0;
} // `car2` goes out of scope, Car is destroyed
```

**Output:**

```
Car Created
Driving...
car1 is now empty
Car Destroyed
```

# Example of shared\_ptr

```
#include <iostream>
#include <memory> // Required for smart pointers
using namespace std;

class Car {
public:
    Car() { cout << "Car Created\n"; }
    ~Car() { cout << "Car Destroyed\n"; }
    void drive() { cout << "Driving...\n"; }
};

int main() {
    shared_ptr<Car> car1 = make_shared<Car>(); // Car is created
    shared_ptr<Car> car2 = car1; // Shared ownership

    car2->drive();

    return 0; // Car is destroyed when last shared_ptr goes out of scope
}
```

**Output:**

```
Car Created
Reference count: 2
Driving...
Car Destroyed
```

# Example of weak\_ptr

```
#include <iostream>
#include <memory>
using namespace std;

class B; // Forward declaration

class A {
public:
    weak_ptr<B> b_ptr;
    // Weak reference to avoid circular dependency
    ~A() { cout << "A Destroyed\n"; }
};

class B {
public:
    shared_ptr<A> a_ptr;
    ~B() { cout << "B Destroyed\n"; }
};
```

```
int main() {
    shared_ptr<A> a = make_shared<A>();
    shared_ptr<B> b = make_shared<B>();

    a->b_ptr = b; // Weak reference (does not increase reference count)
    b->a_ptr = a; // Shared reference (increases reference count)

    return 0; // Both objects are destroyed correctly
}
```

**Output:**

```
B Destroyed
A Destroyed
```

# Three-Way Comparison Operator(< = >)

- The three-way comparison operator (<=>), also called the spaceship operator, was introduced in C++20. It simplifies comparisons by automatically generating ==, <, >, <=, and >= operators.

## What is the <=> Operator?

- The <=> operator **compares two values** and returns:
  - 0 if both are **equal**
  - A **negative value** if the left is **less than** the right
  - A **positive value** if the left is **greater than** the right
- Automatically generates **comparison operators** (==, <, >, <=, >=) when used inside a **class**.

# Example

```
#include <iostream>
#include <compare> // Required for `<=>`
using namespace std;

int main() {
    int a = 5, b = 10;

    // Using the three-way comparison operator
    auto result = a <=> b;

    if (result < 0)
        cout << "a is less than b\n";
    else if (result > 0)
        cout << "a is greater than b\n";
    else
        cout << "a is equal to b\n";

    return 0;
}
```

# Explicit Constructors and Conversion Operators

- C++ allows **implicit** and **explicit** conversions between types.
- However, sometimes **implicit conversions** can cause unexpected behaviors.
- To control this, **explicit constructors** and **explicit conversion operators** are used.

| Feature              | Implicit                                                  | Explicit                                                                                  |
|----------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Constructors         | <code>Number(int)</code> (Allows implicit conversion)     | <code>explicit Number(int)</code> (Prevents implicit conversion)                          |
| Conversion Operators | <code>operator int()</code> (Allows automatic conversion) | <code>explicit operator int()</code> (Requires <code>static_cast&lt;int&gt;(obj)</code> ) |

# Explicit Constructor

- By default, a single-argument constructor can be used for implicit conversion, but marking it as explicit prevents unintended conversions.
- **When to Use explicit?**
  - Use explicit for **constructors with one argument** to prevent **accidental conversions**.
  - Use explicit for **conversion operators** when **implicit conversion could be dangerous**.
  - If **implicit conversion** improves usability, **do not use explicit**.

# Implicit Constructor (Without explicit)

```
#include <iostream>
using namespace std;

class Number {
public:
    int value;

    // Constructor WITHOUT explicit (Allows implicit conversion)
    Number(int v) : value(v) {
        cout << "Constructor called\n";
    }
};

void printNumber(Number n) {
    cout << "Number: " << n.value << endl;
}
```

```
int main() {
    printNumber(10); // Implicit conversion: int → Number
    return 0;
}
```

**Output:**

```
Constructor called
Number: 10
```



# Explicit Constructor (Prevents Implicit Conversion)

```
#include <iostream>
using namespace std;

class Number {
public:
    int value;

    // Constructor WITH explicit (Prevents implicit conversion)
    explicit Number(int v) : value(v) {
        cout << "Constructor called\n";
    }
};

void printNumber(Number n) {
    cout << "Number: " << n.value << endl;
}
```

```
int main() {
    // printNumber(10); // ✗ ERROR: Implicit conversion is blocked

    printNumber(Number(10)); // ✔ OK: Explicit conversion using
    constructor
    return 0;
}
```

**Output:**

```
Constructor called
Number: 10
```

# Conversion Operators

- Conversion operators allow **converting a class object** into another type.
  - **Implicit conversion operators** allow automatic conversion.
  - **Explicit conversion operators** require manual conversion.

# Implicit Conversion Operator

```
#include <iostream>
using namespace std;
```

```
class Number {
public:
    int value;

    Number(int v) : value(v) {}

    // Implicit conversion operator
    operator int() const {
        return value;
    }
};
```

```
int main() {
```

```
    Number num(42);
    int x = num; // Implicitly converts Number → int
```

```
    cout << "Converted int: " << x << endl; // Output: 42
    return 0;
}
```

**Output:**

Converted int: 42

# Explicit Conversion Operator

```
#include <iostream>
using namespace std;

class Number {
public:
    int value;

    Number(int v) : value(v) {}

    // Explicit conversion operator
    explicit operator int() const {
        return value;
    }
};
```

```
int main() {
    Number num(42);

    // int x = num; // ✗ ERROR: Implicit conversion is blocked

    int x = static_cast<int>(num); // ✓ Explicit conversion
    cout << "Converted int: " << x << endl;

    return 0;
}
```

**Output:**

Converted int: 42

# Overloading the Function Call Operator ()

- The function call operator () can be overloaded in C++ to allow an object to be used like a function.
- This is useful for functors (function objects), which are commonly used in STL algorithms, callbacks, and functional programming.

## **Syntax:**

```
class ClassName {  
public:  
    Return Type operator()(ParameterList) {  
        // Function body  
    }  
};
```

# Example

```
#include <iostream>

class Adder {
private:
    int value;
public:
    Adder(int val) : value(val) {}

    // Overloading operator()
    int operator()(int num) {
        return value + num;
    }
};
```

```
int main() {
    Adder add5(5); // Create an object with value 5

    std::cout << "add5(10): " << add5(10) << std::endl; // Calls
    operator(), output: 15

    return 0;
}
```

**Output:**

**add5(10): 15**

# Example

```
#include <iostream>

class Multiplier {
public:
    int operator()(int a, int b) {
        return a * b;
    }
};

int main() {
    Multiplier multiply;
    std::cout << "multiply(4, 5): " << multiply(4, 5) << std::endl; // Output: 20

    return 0;
}
```

**Output:**

```
multiply(4, 5): 20
```

# Example

```
#include <iostream>

class Counter {
private:
    int count;
public:
    Counter() : count(0) {}

    // Overloading operator() to act like a counter
    int operator()() {
        return ++count;
    }
};

int main() {
```

```
    Counter counter;

    std::cout << counter() << std::endl; // Output: 1
    std::cout << counter() << std::endl; // Output: 2
    std::cout << counter() << std::endl; // Output: 3

    return 0;
}
```

**Output:**

1  
2  
3